Visitor Access Management System (VAMS)

Software Requirements Specification

Version 1.0 | October 2026

Table of Contents

1.Introduction **4**
1.1 Purpose **4**
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 Abbreviations 5
2. Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 6
2.3 User Classes and Characteristics 7
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 8
2.6 Assumptions and Dependencies 8
3. External Interface Requirements **9**
3.1 User Interfaces 9
3.2 Hardware Interfaces 9
3.3 Software Interfaces 9
3.4 Communication Interfaces 9
4. System Features **10**
4.1 User Registration 10
4.2 Email Verification 11
4.3 Login 11
4.4 Access Request Approval 12
4.5 Manager Account Creation 13
4.6 QR Badge Generation and Display 13
4.7 QR Code Scanning 14
4.8 Entry/Exit Logging 16
4.9 Real-Time Occupancy Monitoring 16
4.10 Access Validity Management 17
4.11 Password Reset 17
4.12 Logout 18

5. Data Requirements **18**
6. Other Nonfunctional Requirements **19**
6.1 Performance Requirements 19
6.2 Security Requirements 19
6.3 Software Quality Attributes 19
7. Appendix: Glossary **20**
8. References **20**

1.Introduction

1.1 Purpose

This document describes the Visitor Access Management System (VAMS) in detail: the purpose and features of the system, its interfaces, what it will do, the constraints under which it must operate, and how it reacts to external stimuli. It is intended to help developers, testers and end users understand the functional and nonfunctional requirements of VAMS.

1.2 Document Conventions

This SRS follows the IEEE software requirements specification practice. Every requirement has a unique identifier (REQ_01, REQ_02, and so on) and each feature has a priority level (High, Medium or Low). The word “shall” marks a mandatory requirement, and “should” marks a desirable one.

1.3 Intended Audience and Reading Suggestions

- what the final product will do, and can suggest requirements that may have been missed.
- **Project guides and Developers** use this document to understand the technical requirements and turn them into the system design and code.
- **Testers** use it to write unit, integration and system test cases and to understand how the system responds to each stimulus.
- **End users (Owners, Managers, Employees, Guests)** should read the Overall Description to understand **evaluators** can read the whole document to judge the completeness of the analysis.

1.4 Product Scope

VAMS is a web-based, QR-code-driven access control system that replaces paper visitor registers and manual gate-keeping. It manages the complete visitor lifecycle: registration, email verification, manager approval, digital QR badge issuance, gate scanning, entry/exit logging, and real-time occupancy monitoring.

The system is built with Java, Spring Boot, Spring Security, JPA/Hibernate, Thymeleaf and MySQL. It uses the ZXing library to generate QR codes and the html5-qrcode library to scan them in a browser, so no special scanning hardware is needed.

Out of scope for this version: biometric authentication, GPS tracking, integration with turnstiles or electronic locks, native iOS/Android apps, SMS/push notifications, multi-building deployment, video surveillance integration, and automated threat detection.

1.5 Abbreviations

- **VAMS**: Visitor Access Management System
- **RBAC**: Role-Based Access Control
- **QR**: Quick Response (code)
- **UUID**: Universally Unique Identifier
- **JPA**: Java Persistence API
- **ORM**: Object-Relational Mapping
- **MVC**: Model View Controller
- **CSRF**: Cross-Site Request Forgery
- **SRS**: Software Requirements Specification

2. Overall Description

2.1 Product Perspective

VAMS is a new, self-contained web application. Users reach it through a web browser over the intranet or internet. The application server talks to a MySQL database, and a

cooperating external service (Firebase Authentication) handles email verification and password-reset links. Figure 1 shows the system environment.

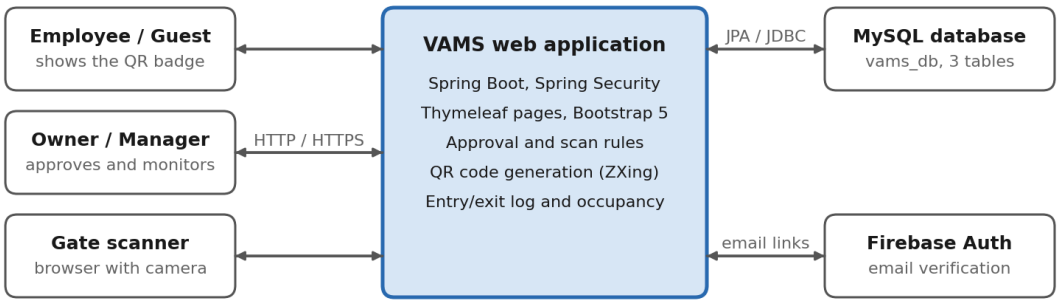


Figure 1. System environment

2.2 Product Functions

The major functions of VAMS are summarised below. Section 4 describes each one in detail.

- The system shall allow a person to register as a Manager, Employee or Guest.
- The system shall verify the user's email address through Firebase.
- The system shall let a user log in and log out securely.
- The system shall automatically create a PENDING access request for every registration.
- The system shall let a Manager or Owner approve or reject access requests, following a role hierarchy.
- The system shall let the Owner create Manager accounts directly.
- The system shall generate a unique QR badge for every approved user.
- The system shall let a gate operator scan QR badges using a browser camera.
- The system shall automatically decide whether a scan is an ENTRY or an EXIT.
- The system shall keep an append-only log of all scan events.
- The system shall compute and display live building occupancy.
- The system shall support ONE_DAY, UNTIL_LOGOFF and ONGOING access validity.
- The system shall let users reset a forgotten password.

2.3 User Classes and Characteristics

Owner (Super Administrator). Has full control of the system. Can create Manager accounts, approve or reject requests from every role, and view all users, all requests and the live occupancy.

Manager (Facility Manager). Reviews and approves or rejects requests from Employees and Guests only. Views occupancy and statistics, and operates the gate scanner. A Manager must first be approved by the Owner.

Employee (Regular Staff). Registers, waits for approval, and then uses a personal QR badge for daily entry and exit.

Guest (Visitor or Temporary Staff). Registers with a purpose of visit, waits for approval, and then uses a QR badge whose validity may be limited in time.

Scanner Operator. A role performed by a Manager or Owner who runs the QR scanning station at a gate.

2.4 Operating Environment

The server application runs on JDK 17 with Spring Boot, uses MySQL 8.0 or later as its database, and is built with Apache Maven. The system can be hosted on Windows, Linux or macOS. Clients need only a modern web browser (Chrome 90+, Firefox 88+, Edge 90+ or Safari 14+). Scanner stations need a device with a camera of at least 2 MP.

The system follows a layered architecture, shown in Figure 2: Presentation, Security, Controller, Service, Persistence and Database layers, using the MVC pattern.

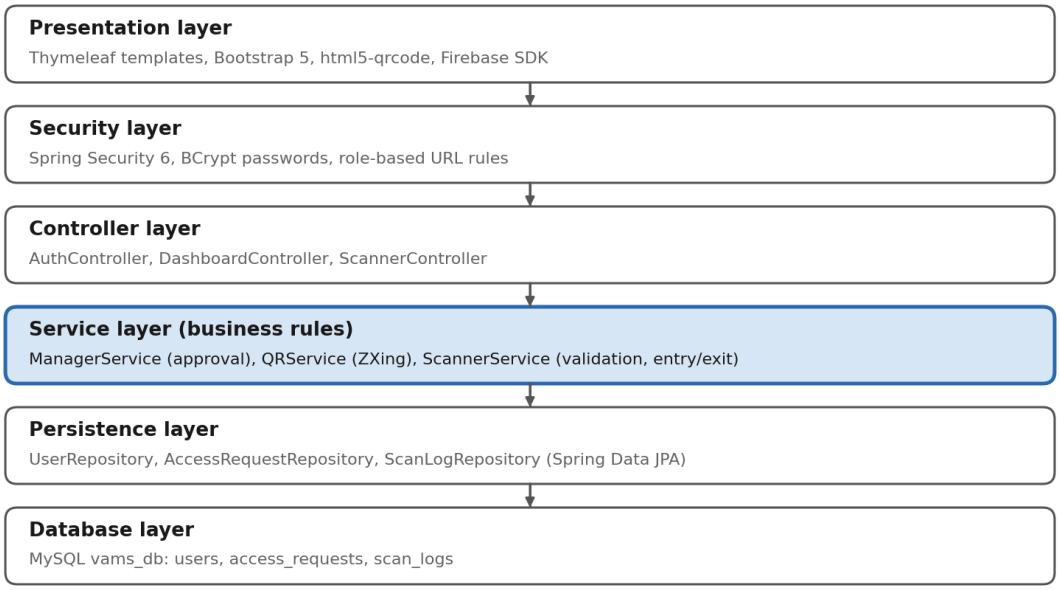


Figure 2. Layered architecture

2.5 Design and Implementation Constraints

- The system must be written in Java using the Spring Boot framework.
- Passwords must be stored only as BCrypt hashes.
- The scan log table must be append-only: no update or delete operations may be exposed.
- QR tokens must be UUID version 4 values.
- The user interface must be server-rendered with Thymeleaf and styled with Bootstrap 5.

2.6 Assumptions and Dependencies

- Gate devices have network access to the VAMS server.
- Users have a valid email address that they can access.
- The Firebase Authentication service is available for email verification and password reset.
- The server clock is correct, because all timestamps and expiry checks rely on it.

3. External Interface Requirements

3.1 User Interfaces

The interface is a responsive web UI that works from 320 px screen width upward. It consists of these screens: Login, Registration, Email Verification, Forgot Password, Owner Dashboard, Manager Dashboard, Pending Requests, Guest/Employee Dashboard (QR badge), Pending Approval, Scanner, and Scan Result.

3.2 Hardware Interfaces

- **Server:** minimum dual-core 2.0 GHz processor, 4 GB RAM, 20 GB storage, 100 Mbps network.
- **Scanner station:** any smartphone, tablet, laptop or desktop with a rear camera or webcam of at least 2 MP.
- **User device:** any device with a modern browser.

3.3 Software Interfaces

- **MySQL database (vams_db):** accessed through Spring Data JPA, Hibernate and the MySQL Connector/J JDBC driver.
- **Firebase Authentication SDK:** used for email verification links and password-reset links.
- **ZXing library:** used on the server to generate QR code images.
- **html5-qrcode library:** used in the browser to read QR codes from the camera.

3.4 Communication Interfaces

All client-server communication uses HTTP/HTTPS. The scanner submits the decoded token with an HTTP POST request to /scanner/verify. Forms are protected by CSRF tokens.

4. System Features

This section describes the functional requirements feature by feature. It is written mainly for developers.

4.1 User Registration

Description and Priority. A new person registers by giving a username, full name, email, password, role (Manager, Employee or Guest) and, for guests, a purpose of visit. Priority: High.

Stimulus/Response Sequence

1. The user opens the registration page and submits the form.
2. The system checks that the username and email are not already used.
3. The system hashes the password with BCrypt and saves the user.
4. The system creates an access request with status PENDING.
5. The system redirects the user to email verification.

Functional Requirements

- REQ_01: The system shall allow a visitor to submit a registration form.
- REQ_02: The system shall reject a registration whose username already exists and show the message “Username already exists!”.
- REQ_03: The system shall reject a registration whose email already exists and show the message “Email already registered!”.
- REQ_04: The system shall default the role to GUEST if none is selected.
- REQ_05: The system shall store the password only as a BCrypt hash with automatic salt.
- REQ_06: The system shall automatically create an access request with status PENDING for every new registration.

4.2 Email Verification

Description and Priority. The system confirms that the registered email address is real by sending a link through Firebase Authentication. Priority: High.

Stimulus/Response Sequence

1. The user enters their email on the verification page.
2. Firebase sends a sign-in link to that email.
3. The user clicks the link and Firebase authenticates it.
4. The system redirects the user to the login page.

Functional Requirements

- REQ_07: The system shall send a verification link to the user's email through Firebase.
- REQ_08: The system shall redirect the user to the login page after successful verification.

4.3 Login

Description and Priority. Users authenticate with username and password and are sent to a dashboard based on their role. Priority: High.

Stimulus/Response Sequence

1. The user opens the login page and enters credentials.
2. The system loads the user record and compares the password with the stored BCrypt hash.
3. On success, the system creates a session and routes the user: Owner to the Owner Dashboard, Manager to the Manager Dashboard, others to the Guest/Employee Dashboard.
4. On failure, the system shows "Invalid username or password."
5. If the user has no approved access request, the system shows the Pending Approval page instead of the dashboard.

Functional Requirements

- REQ_09: The system shall authenticate users with username and password.
- REQ_10: The system shall route each user to the dashboard that matches their role.
- REQ_11: The system shall block users without an APPROVED request from using the dashboard and show the Pending Approval page.
- REQ_12: The system shall restrict URLs by role: /owner/** to the Owner only, and /manager/** and /scanner/** to Managers and the Owner.

4.4 Access Request Approval

Description and Priority. Every access request goes through a hierarchical review. The approval hierarchy is shown in Figure 3. Priority: High.

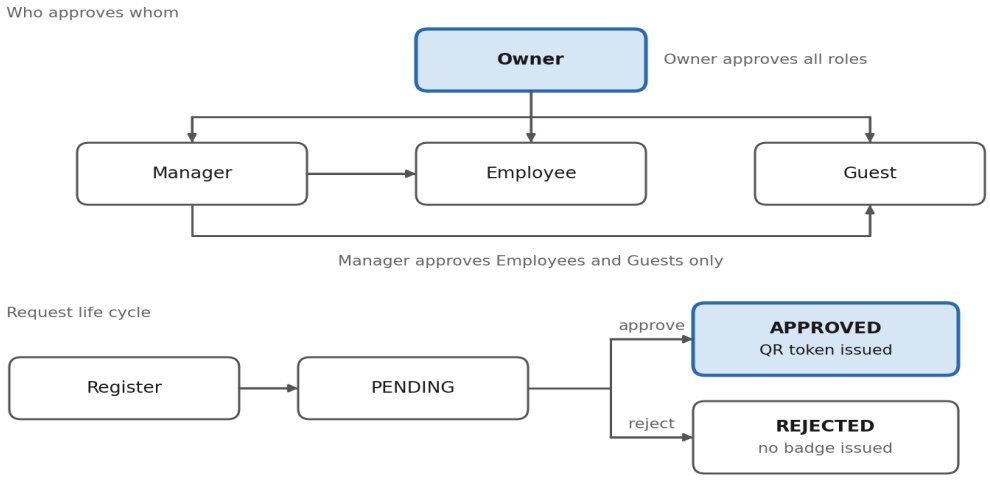


Figure 3. Approval hierarchy and request life cycle

Stimulus/Response Sequence

1. A Manager or Owner opens the list of pending requests.
2. The system shows only the requests the user is allowed to decide.
3. The approver clicks Approve or Reject.
4. On approval, the system sets the access level and validity, records the approval time and generates a QR token.
5. On rejection, the system sets the status to REJECTED.

Functional Requirements

- REQ_13: The system shall let the Owner approve or reject requests from all roles, including Managers.
- REQ_14: The system shall let a Manager approve or reject requests from Employees and Guests only, and shall hide Manager and Owner requests from Managers.
- REQ_15: When the Owner approves a request, the system shall set the access level to RESTRICTED and the validity to ONGOING.
- REQ_16: When a Manager approves a request, the system shall set the access level to PUBLIC and the validity to ONGOING.
- REQ_17: On approval, the system shall record the approval timestamp and generate a unique UUID v4 QR token.
- REQ_18: The system shall require a Manager to be approved by the Owner before the Manager can use the Manager Dashboard.

4.5 Manager Account Creation

Description and Priority. The Owner can create a Manager account directly, without the normal registration flow. Priority: High.

Functional Requirements

- REQ_19: The system shall let the Owner create a Manager account by entering a username and password on the Owner Dashboard.
- REQ_20: The system shall store the new Manager's password as a BCrypt hash and assign the MANAGER role.

4.6 QR Badge Generation and Display

Description and Priority. Every approved user receives a digital QR badge that encodes their unique token. Priority: High.

Stimulus/Response Sequence

1. An approved user logs in and opens the dashboard.
2. The system reads the user's QR token.
3. The system encodes the token into a 250 x 250 pixel PNG QR image and converts it to Base64.
4. The dashboard shows the QR image, the token text, the status (ACTIVE or PENDING), the access level and the validity type.

Functional Requirements

- REQ_21: The system shall generate a QR code image of 250 x 250 pixels from the user's token using the ZXing library.
- REQ_22: The system shall embed the QR image in the page as a Base64-encoded PNG.
- REQ_23: The system shall show the badge status, access level and validity type next to the QR code.
- REQ_24: The system shall ensure that every QR token is unique.

4.7 QR Code Scanning

Description and Priority. A gate operator scans a user's QR badge using any browser-enabled device with a camera. Priority: High.

Stimulus/Response Sequence

1. The operator opens the scanner page and presses "Start Scanner".
2. The camera starts and detects the QR code.
3. The decoded token is submitted automatically to the server.
4. The server validates the token (see Figure 4) and returns a PASS or FAIL result page.

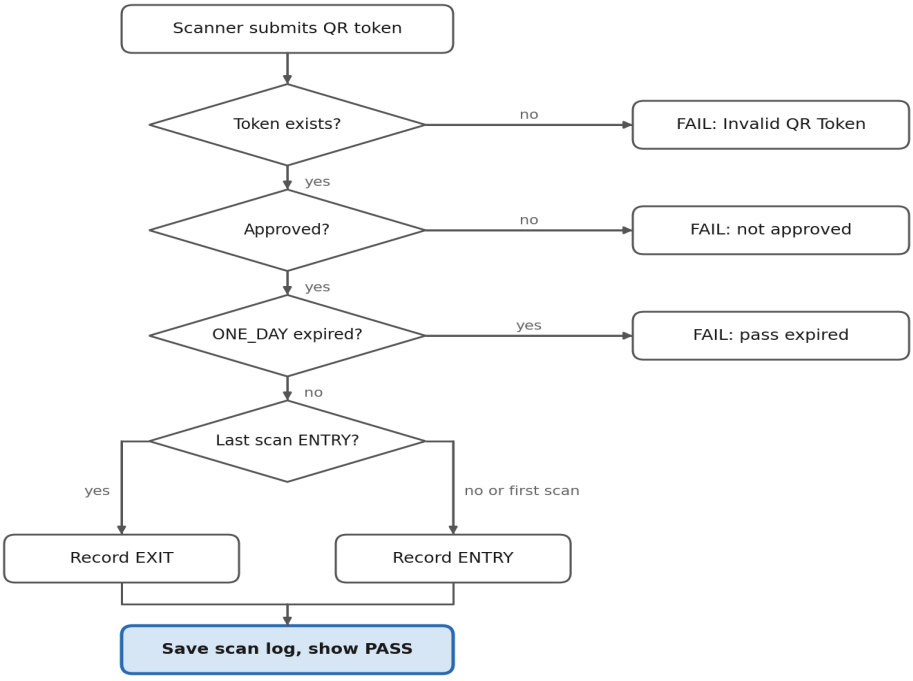


Figure 4. Scan validation flow

Functional Requirements

- REQ_25: The system shall provide a scanner page that uses the rear camera and detects QR codes automatically.
- REQ_26: The system shall provide a manual text field for entering a token when the camera cannot be used.
- REQ_27: The system shall fail the scan with the reason “Invalid QR Token” if the token is not found.
- REQ_28: The system shall fail the scan with the reason “Request Status is: [status]” if the request is not APPROVED.
- REQ_29: The system shall fail the scan with the reason “Expired (One-Day Pass Ended)” if a ONE_DAY pass is more than 24 hours past its approval time.
- REQ_30: The system shall show a green PASS banner with the scan direction for a valid scan and a red FAIL banner with the reason for an invalid scan.
- REQ_31: The system shall record the gate location with each scan.

4.8 Entry/Exit Logging

Description and Priority. The system decides automatically whether a valid scan is an ENTRY or an EXIT and writes it to an immutable log. Priority: High.

Stimulus/Response Sequence

1. A valid token is received.
2. The system loads the user's previous scans.
3. If there is no previous scan, or the last scan was EXIT, the new scan is ENTRY. If the last scan was ENTRY, the new scan is EXIT.
4. The system saves a scan log entry with the user, gate, type and timestamp.

Functional Requirements

- REQ_32: The system shall treat the first scan of a user as ENTRY and shall toggle between ENTRY and EXIT on every following valid scan.
- REQ_33: The system shall save, for every valid scan, the user, gate location, scan type and a timestamp with microsecond precision.
- REQ_34: The system shall not provide any update or delete operation on scan logs.

4.9 Real-Time Occupancy Monitoring

Description and Priority. Owner and Manager dashboards show how many people are currently inside the premises. Priority: High.

Live occupancy is calculated as: Occupancy = max(0, total ENTRY scans − total EXIT scans).

Functional Requirements

- REQ_35: The system shall calculate live occupancy from all scan logs using the formula above, never showing a negative value.
- REQ_36: The system shall display live occupancy on the Owner Dashboard and the Manager Dashboard.

- REQ_37: The system shall show the Manager the number of Employees, Managers and Guests, and the number of PUBLIC and RESTRICTED access grants.
- REQ_38: The Owner Dashboard shall list all users with their roles and all requests with their status, with Approve and Reject buttons for pending requests.

4.10 Access Validity Management

Description and Priority. An access pass can have different validity policies. Priority: Medium.

- **ONE_DAY:** expires automatically 24 hours after approval.
- **UNTIL_LOGOFF:** valid for the current session until the user logs out.
- **ONGOING:** valid until an administrator revokes it.

Functional Requirements

- REQ_39: The system shall support the validity types ONE_DAY, UNTIL_LOGOFF and ONGOING.
- REQ_40: The system shall enforce ONE_DAY expiry at scan time without manual action.
- REQ_41: The system shall support the access levels PUBLIC and RESTRICTED.

4.11 Password Reset

Description and Priority. A user who forgot a password can set a new one after email verification. Priority: Medium.

Functional Requirements

- REQ_42: The system shall let a user request a password reset by entering their registered email.
- REQ_43: The system shall verify the email through a Firebase link before allowing a new password to be set.
- REQ_44: The system shall store the new password as a BCrypt hash and redirect to the login page.

4.12 Logout

Description and Priority. A logged-in user can end the session. Priority: High.

Functional Requirements

- REQ_45: The system shall let a logged-in user log out.
- REQ_46: On logout, the system shall invalidate the session and remove the session data.
- REQ_47: The user shall be able to log out from any screen.

5. Data Requirements

The database vams_db has three tables, shown in Figure 5.

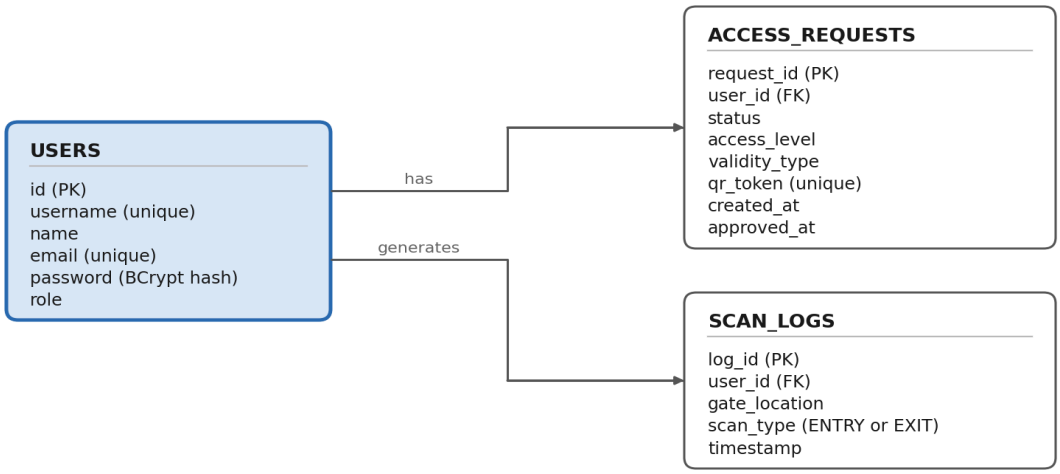


Figure 5. Entity-relationship diagram

- **users** holds id, username (unique), name, email (unique), password (BCrypt hash) and role (OWNER, MANAGER, EMPLOYEE or GUEST).
- **access_requests** holds request_id, user_id (foreign key), status (PENDING, APPROVED or REJECTED), access_level (PUBLIC or RESTRICTED), validity_type, qr_token (unique), created_at and approved_at.
- **scan_logs** holds log_id, user_id (foreign key), gate_location, scan_type (ENTRY or EXIT) and timestamp.

One user can have many access requests and many scan logs. The scan_logs table is append-only.

- REQ_48: The system shall enforce unique constraints on username, email and qr_token.
- REQ_49: The system shall use foreign keys from access_requests and scan_logs to users.
- REQ_50: The system shall store timestamps with microsecond precision (DATETIME(6)).

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- REQ_51: A QR scan shall be validated and the result shown in less than 2 seconds under normal load.
- REQ_52: Dashboard pages should load in less than 3 seconds.

6.2 Security Requirements

- REQ_53: The system shall hash all passwords with BCrypt.
- REQ_54: The system shall protect all state-changing forms with CSRF tokens.
- REQ_55: The system shall deny access to protected URLs for users without the required role and return HTTP 403.
- REQ_56: The system shall destroy all session data after logout.
- REQ_57: QR tokens shall be random UUID v4 values that cannot be predicted.

6.3 Software Quality Attributes

- **Usability:** the interface is responsive and works on phones, tablets and desktops without installation.
- **Availability:** the system should be available whenever the premises are open.
- **Reliability:** database transactions keep approval and scan data consistent.

- **Maintainability:** the layered design and clean separation of controllers, services and repositories make the code easy to change.
- **Scalability:** the scan endpoint is stateless, so the application can be scaled and moved to cloud hosting.
- **Auditability:** every scan is traceable to a user, gate, direction and time.
- **Portability:** the application runs on any operating system that supports JDK 17.

7. Appendix: Glossary

- **Access request:** a record created at registration that must be approved before a user can use the system.
- **Badge:** the QR code shown on the user's dashboard that acts as a digital access card.
- **Gate:** a physical entry or exit point where a scanner station is placed.
- **Occupancy:** the number of people currently inside the premises.
- **Spring Boot:** a Java framework for building web applications.
- **Thymeleaf:** a server-side template engine that renders HTML pages.
- **BCrypt:** a one-way password hashing algorithm with built-in salt.
- **UUID v4:** a 128-bit random identifier.
- **ZXing:** an open-source library for generating and reading barcodes.

8. References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- Spring Boot, Spring Security and Spring Data JPA reference documentation.
- ZXing (Zebra Crossing) barcode library, version 3.5.3.
- html5-qrcode library documentation.
- Firebase Authentication documentation.